



Institut Universitaire des sciences

- Faculté des sciences et de technologie

✓ TD N°3 Modélisation

Nom & Prénom : COFFY Cliford

Niveau : L3

Prof. : Ismael SAINT AMOUR

Date : 15/04/2026

TD1 — Réseau social intelligent

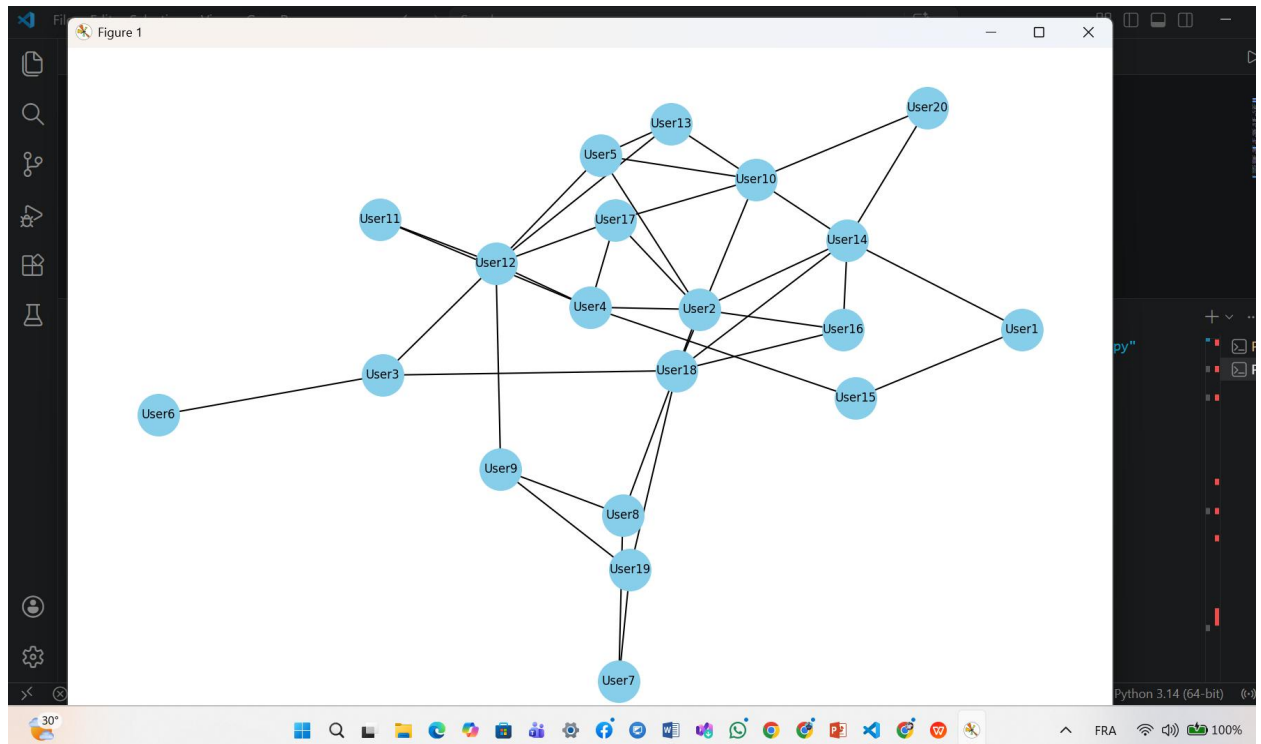
Construire un graphe représentant des utilisateurs :

- sommets = utilisateurs
- arêtes = relations

Objectifs

1. créer un réseau de 20 utilisateurs
2. trouver les personnes les plus influentes
3. calculer le degré de chaque nœud
4. détecter les communautés

1. créer un réseau de 20 utilisateurs



```
File Edit Selection View Go Run ... Search
import networkx as nx.py 3 X
C: > Users > User > Documents > import networkx as nx.py > ...
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 from networkx.algorithms import community
4
5 # === 1. Création du graphe ===
6 G = nx.Graph()
7
8 # Ajouter 20 utilisateurs
9 for i in range(1, 21):
10     G.add_node(f"User{i}")
11
12 # Générer quelques relations (arêtes)
13 relations = [
14     ("Ismael", "Cliford"), ("Marie-france", "Fritz"), ("Erntz", "Charles"),
15     ("Christy", "Jean"), ("Jeanne", "COFFY"), ("Paul", "Wisdaelle"),
16     ("Wisner", "Clif"), ("Vanessa", "Janette"), ("Judes", "Paulette"),
17     ("Pierre", "Judithe"), ("Ruth", "Bellevue"), ("Desir", "Noel"),
18     ("Marie-france", "Erntz"), ("Fritz", "UCharles"), ("Jeanne", "COFFY"),
19     ("Clif", "Vanessa"), ("Janette", "Judes"), ("Paulette", "Pierre"),
20 ]
21 G.add_edges_from(relations)
22
23 # === 2. Visualisation du graphe ===
24 plt.figure(figsize=(8, 6))
25 nx.draw(G, with_labels=True, node_color="lightblue", font_weight="bold")
26 plt.title("Réseau social de 20 utilisateurs")
27 plt.show()
28
29 # === 3. Calcul du degré de chaque nœud ===
```

2. trouver les personnes les plus influentes

```
10 # Ajouter des relations aléatoires

PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/files a telecharger/TD2m.py"
User4 avec 5 connexions
User18 avec 5 connexions

Top utilisateurs influents (par centralité) :
User12 avec centralité 0.237
User14 avec centralité 0.183
User2 avec centralité 0.176
User18 avec centralité 0.173
User3 avec centralité 0.120

Degré de chaque utilisateur :
User1 : 2
User2 : 6
User3 : 3
User4 : 5
User5 : 4
User6 : 1
User7 : 2
```

```
File Edit Selection View Go Run ... Search

import networkx as nx.py 3 X
C:\Users\User> Documents > import networkx as nx.py > ...

33 # Utilisateur le plus influent (celui avec le plus de connexions)
34 influential = max(degrees, key=degrees.get)
35 print("Utilisateur le plus influent :", influential)
36
37 # == 4. Détection des communautés ==
...

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/import networkx as nx.py"
User1 → degré 0
User2 → degré 0
User3 → degré 0
User4 → degré 0
User5 → degré 0
User6 → degré 0
User7 → degré 0
User8 → degré 0
User9 → degré 0
User10 → degré 0
User11 → degré 0
User12 → degré 0
User13 → degré 0
User14 → degré 0
User15 → degré 0
User16 → degré 0
User17 → degré 0
User18 → degré 0
User19 → degré 0
User20 → degré 0
Utilisateurs les plus influents : ['Marie-france', 'Fritz', 'Erntz', 'Clif', 'Vanessa', 'Janette', 'Judes', 'Paulette', 'Pierre']
PS C:\Users\User>
```

3. calculer le degré de chaque nœud

```
8 G.add_nodes_from(users)
9
10 # Ajouter des relations aléatoires

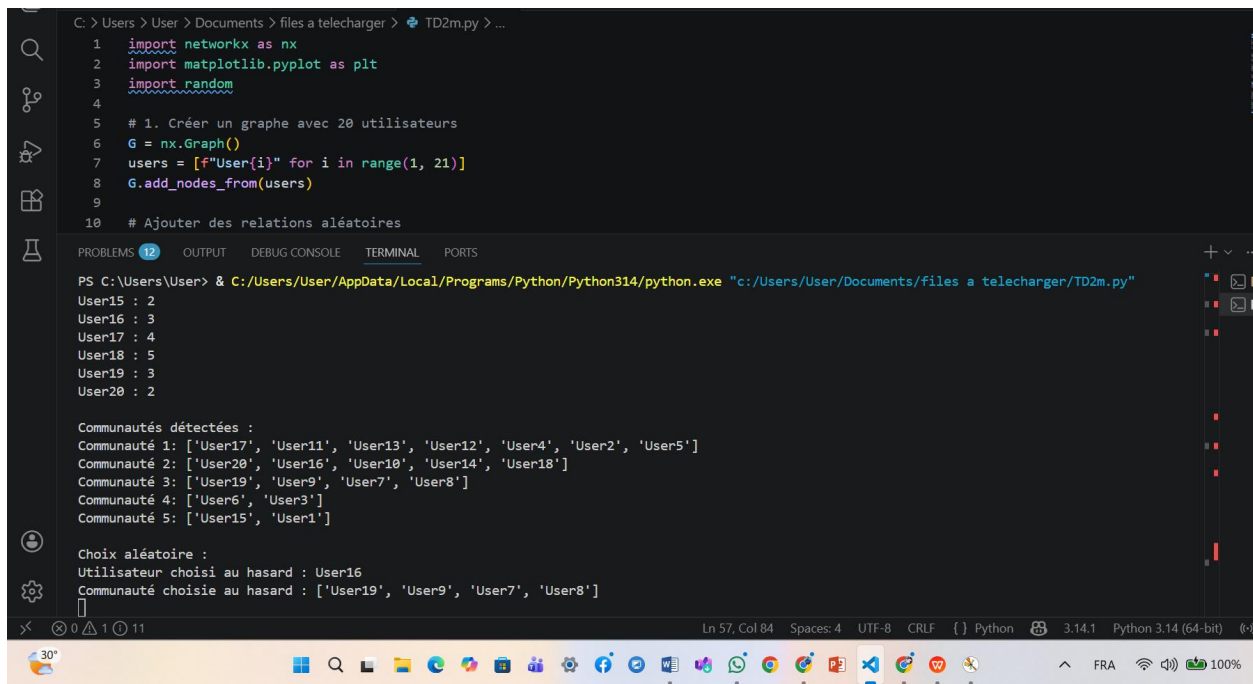
PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/files a telecharger/TD2m.py"

Degré de chaque utilisateur :
User1 : 2
User2 : 6
User3 : 3
User4 : 5
User5 : 4
User6 : 1
User7 : 2
User8 : 3
User9 : 3
User10 : 4
User11 : 2
User12 : 7
User13 : 3
User14 : 6
User15 : 2
User16 : 3
```

```
File Edit Selection View Go Run ... Search
import networkx as nx.py 3 X
C:\> Users > User > Documents > import networkx as nx.py > ...
37 # === 4. Détection des communautés ===
38 communities = community.girvan_newman(G)
39 first_level = next(communities)
40 print("Communautés détectées :", [list(c) for c in first_level])
41

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users
Erntz a un degré de 2
Charles a un degré de 1
Christy a un degré de 1
Jean a un degré de 1
Jeanne a un degré de 1
COFFY a un degré de 1
Paul a un degré de 1
Wisdaelle a un degré de 1
Wisner a un degré de 1
Clif a un degré de 2
Vanessa a un degré de 2
Janette a un degré de 2
Judes a un degré de 2
Paulette a un degré de 2
Pierre a un degré de 2
Judithe a un degré de 1
Ruth a un degré de 1
Bellevue a un degré de 1
Desir a un degré de 1
Noel a un degré de 1
UCharles a un degré de 1
PS C:\Users\User>
```

4. détecter les communautés



```
C: > Users > User > Documents > files a telecharger > TD2m.py > ...
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 import random
4
5 # 1. Créer un graphe avec 20 utilisateurs
6 G = nx.Graph()
7 users = [f"User{i}" for i in range(1, 21)]
8 G.add_nodes_from(users)
9
10 # Ajouter des relations aléatoires

PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/files a telecharger/TD2m.py"
User15 : 2
User16 : 3
User17 : 4
User18 : 5
User19 : 3
User20 : 2

Communautés détectées :
Communauté 1: ['User17', 'User11', 'User13', 'User12', 'User4', 'User2', 'User5']
Communauté 2: ['User20', 'User16', 'User10', 'User14', 'User18']
Communauté 3: ['User19', 'User9', 'User7', 'User8']
Communauté 4: ['User6', 'User3']
Communauté 5: ['User15', 'User1']

Choix aléatoire :
Utilisateur choisi au hasard : User16
Communauté choisie au hasard : ['User19', 'User9', 'User7', 'User8']
```

TD2 — Chaîne de Markov (météo + économie)

Modéliser deux systèmes :

1. météo

- Soleil
- Pluie
- Nuage

2. économie

- Hausse
- Stable

Objectifs

1. construire la matrice de transition
2. simuler 50 à 100 jours
3. calculer la distribution stationnaire
4. comparer simulation vs théorie

1. construire la matrice de transition

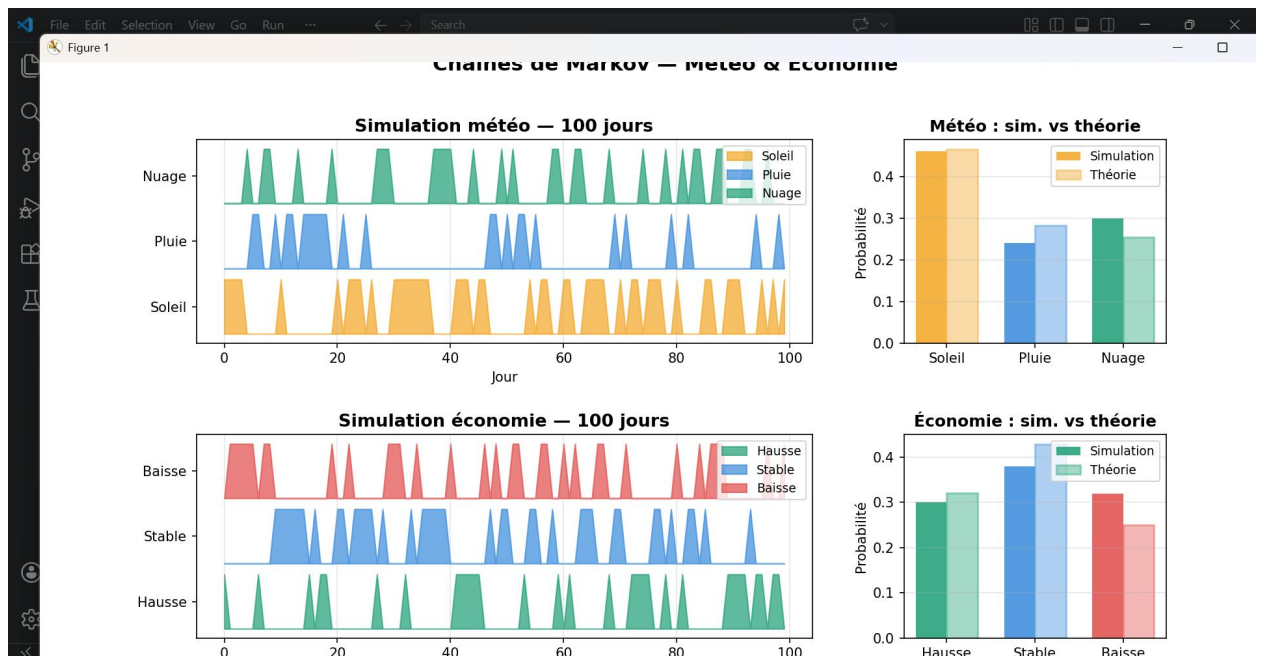
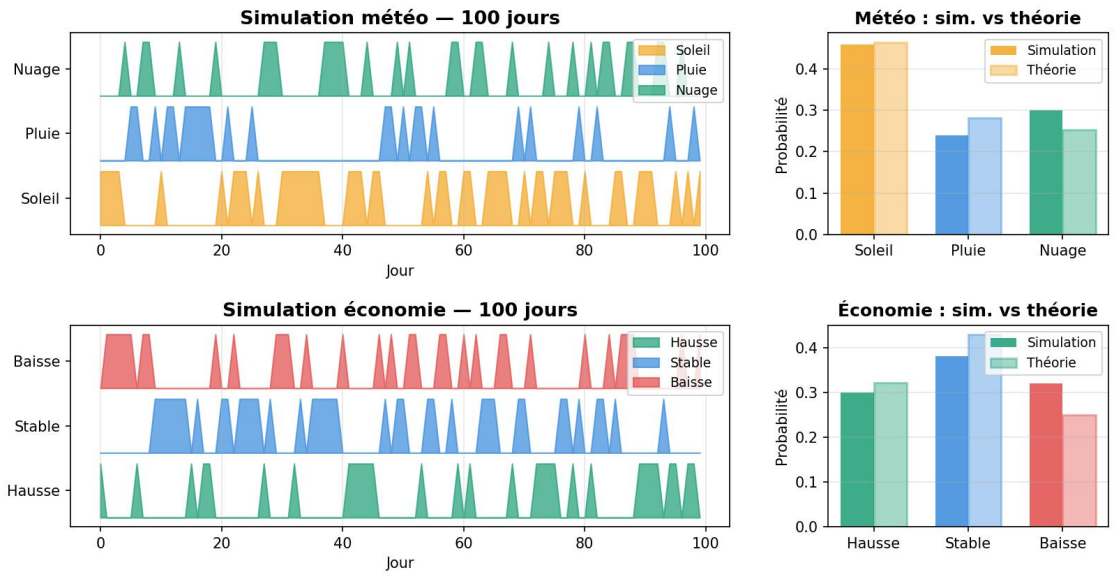


Figure 1

Chaines de Markov — Météo & Economie



```

62
63
64 def empirical_distribution(history, states):
PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/files a telecharger/TD2m.py"
KeyboardInterrupt
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/files a telecharger/TD2m.py"
Simulation météo (10 premiers jours) : ['Soleil', 'Pluie', 'Soleil', 'Soleil', 'Soleil', 'Soleil', 'Soleil', 'Soleil', 'Pluie', 'Pluie']
Simulation économie (10 premiers jours) : ['Hausse', 'Stable', 'Stable', 'Stable', 'Stable', 'Baisse', 'Baisse', 'Hausse', 'Hausse', 'Stable']

Distribution stationnaire météo : {'Soleil': np.float64(0.46478873239436624), 'Pluie': np.float64(0.28169014084507044), 'Nuage': np.float64(0.25352112676056343)}
Distribution stationnaire économie : {'Hausse': np.float64(0.3214285714285715), 'Stable': np.float64(0.42857142857142855), 'Baisse': np.float64(0.25)}

Comparaison météo :
Simulation : {'Soleil': 0.49, 'Pluie': 0.31, 'Nuage': 0.2}
Théorie : {'Soleil': np.float64(0.46478873239436624), 'Pluie': np.float64(0.28169014084507044), 'Nuage': np.float64(0.25352112676056343)}

Comparaison économie :
Simulation : {'Hausse': 0.3, 'Stable': 0.45, 'Baisse': 0.25}
Théorie : {'Hausse': np.float64(0.3214285714285715), 'Stable': np.float64(0.42857142857142855), 'Baisse': np.float64(0.25)}
PS C:\Users\User>

```

2. simuler 50 à 100 jours


```
File Edit Selection View Go Run ... Search
import networkx as nx.py 6 TD2.py 1 X
C:\Users\User> Documents > files a telecharger > TD2.py > ...
62
63 print("\nComparaison météo :")
64 print("Simulation :", freq_weather)
65 print("Théorie :", pi_weather)
66
67 print("\nComparaison économie :")
68 print("Simulation :", freq_economy)
69 print("Théorie :", pi_economy)
70

PROBLEMS 20 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/files a telecharger/TD2.py"
sse', 'Soleil, Stable', 'Pluie, Baisse', 'Pluie, Stable', 'Pluie, Stable', 'Nuage, Hausse', 'Soleil, Hausse', 'Soleil, Hausse', 'Soleil, Stable', 'Nuage, Stable',
'Nuage, Baisse', 'Soleil, Stable', 'Soleil, Stable', 'Soleil, Baisse', 'Soleil, Baisse', 'Soleil, Hausse', 'Pluie, Hausse', 'Nuage, Hausse', 'Nuage, Hausse', 'Nuag
e, Baisse', 'Nuage, Baisse', 'Pluie, Hausse', 'Nuage, Hausse', 'Pluie, Hausse']
Distribution stationnaire météo : [0.46031746 0.31746032 0.22222222]
Distribution stationnaire économie : [0.45652174 0.2826087 0.26086957]

Comparaison météo :
Simulation : [0.52941176 0.2745098 0.19607843]
Théorie : [0.46031746 0.31746032 0.22222222]

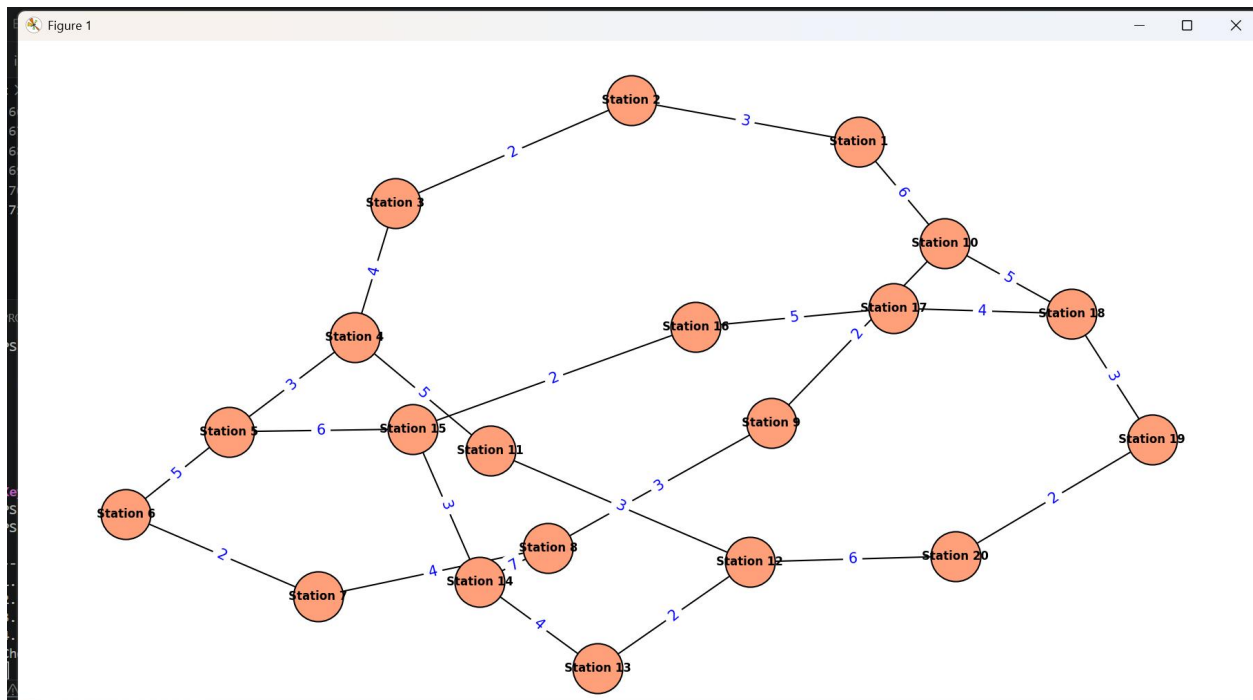
Comparaison économie :
Simulation : [0.39215686 0.25490196 0.35294118]
Théorie : [0.45652174 0.2826087 0.26086957]
PS C:\Users\User>
```

TD3 — Réseau de transport (type métro)

Créer un réseau de métro simplifié.

Objectifs

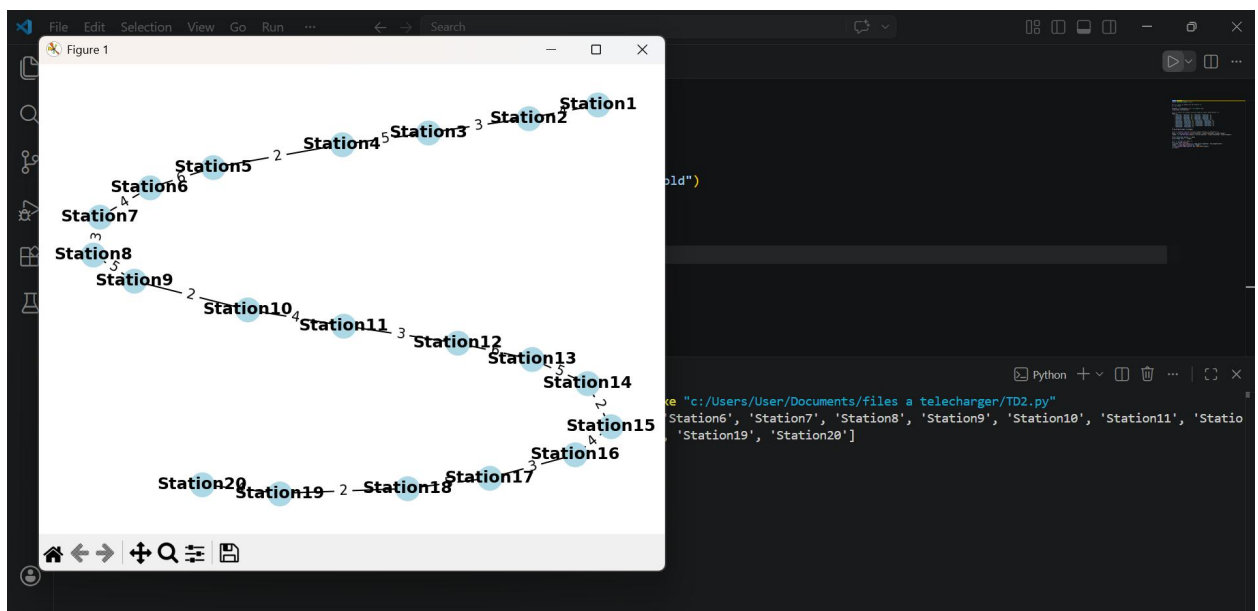
1. créer 15 à 25 stations
2. relier les stations
3. associer des temps de trajet
4. calculer itinéraires optimaux
5. afficher les lignes



```
C: > Users > User > Documents > files a telecharger > TD2.py > ...
55     print(f"Temps de trajet estimé : {temps_total} minutes")
56 except:
57     print("\nErreur : Vérifiez les numéros des stations.")
58 elif choix == "4":
59     print("\nMerci d'avoir utilisé ce programme. À bientôt !")
60     break
61 else:
62     print("\nOption invalide, veuillez réessayer.")
63
```

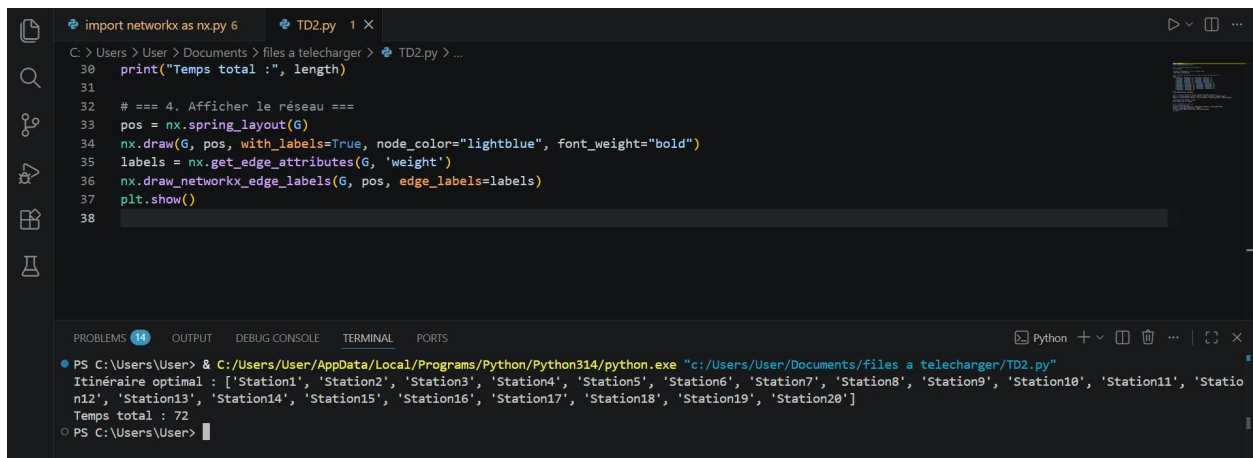
PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/files a telecharger/TD2.py"
Station 5 <-> Station 6 : 5 min
Station 5 <-> Station 20 : 4 min
Station 6 <-> Station 7 : 2 min
Station 7 <-> Station 8 : 4 min
Station 8 <-> Station 9 : 3 min
Station 8 <-> Station 15 : 7 min
Station 9 <-> Station 10 : 2 min
Station 11 <-> Station 12 : 3 min
Station 11 <-> Station 20 : 6 min
Station 12 <-> Station 13 : 2 min
Station 13 <-> Station 14 : 4 min
Station 14 <-> Station 15 : 3 min
Station 15 <-> Station 16 : 2 min
Station 16 <-> Station 17 : 5 min
Station 17 <-> Station 18 : 4 min
Station 18 <-> Station 19 : 3 min
Station 19 <-> Station 20 : 2 min
```



4. calculer itinéraires optimaux

5. afficher les lignes



The image shows a Visual Studio Code editor window with a Python script named `TD2.py` and its terminal output. The script calculates an optimal route and displays the network graph. The terminal output shows the optimal route and the total time.

```
import networkx as nx
30 print("Temps total :", length)
31
32 # === 4. Afficher le réseau ===
33 pos = nx.spring_layout(G)
34 nx.draw(G, pos, with_labels=True, node_color="lightblue", font_weight="bold")
35 labels = nx.get_edge_attributes(G, 'weight')
36 nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
37 plt.show()
38
```

Terminal Output:

```
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/User/Documents/files a telecharger/TD2.py"
Itinéraire optimal : ['Station1', 'Station2', 'Station3', 'Station4', 'Station5', 'Station6', 'Station7', 'Station8', 'Station9', 'Station10', 'Station11', 'Station12', 'Station13', 'Station14', 'Station15', 'Station16', 'Station17', 'Station18', 'Station19', 'Station20']
Temps total : 72
PS C:\Users\User>
```